1: Title

Healthcare AI Chatbot
AI Engineer Technical Assignment

Overview
A production-ready, intelligent healthcare assistant built to provide accurate, evidence-based health information while ensuring strict medical safety guardrails.

Key Capabilities

- Multi-LLM Support (Google Gemini 2.5 Flash & Groq Llama 3.3)
- RAG-powered knowledge retrieval
- Real-time safety guardrails (Emergency, Diagnosis, Prescription)

2: System Architecture

(Diagram placeholder - You can insert the flowchart from the README here)

Core Components:

1. **Frontend**: Streamlit UI for an interactive, conversational experience.
2. **Safety Layer**: Pre-generation (Input checking) and Post-generation (Output validation) guardrails.
3. **Retrieval-Augmented Generation (RAG)**: FAISS Vector Database + HuggingFace Embeddings.
4. **Orchestrator**: LangChain pipelines connecting memory, prompts, and the active LLM.
5. **LLM Engine**: Interchangeable LLM factory supporting Gemini and Groq via LangChain.

3: Tech Stack & LLM Rationale

Technology Stack:

- **Language**: Python 3.11+
- **Framework**: Streamlit & LangChain
- **Embeddings**: Sentence-transformers (`all-MiniLM-L6-v2`) via HuggingFace
- **Vector DB**: FAISS (Fast, offline, and lightweight)
- **Document Parsing**: PyMuPDF (Handles layout, columns, and tables efficiently)

LLM Selection & Rationale:

- **Google Gemini 2.5 Flash**: Chosen for its exceptional speed, massive context window, and strong reasoning capabilities. Ideal as the default model.
- **Groq (Llama 3.3)**: Integrated as an alternative to demonstrate architectural flexibility and multi-provider capability, offering ultra-low latency inference.

4: Prompt Engineering & Workflow

Prompt Engineering Strategy:

- **System Identity**: Strictly defined as an educational assistant, not a doctor.
- **Context Injection**: RAG chunks are injected as simulated "Human -> AI" conversation turns to naturally ground the model.
- **Personalization**: User health profiles (Age, Conditions) are dynamically injected into the system prompt with explicit instructions to personalize without diagnosing.

Application Workflow:

1. User submits query -> Sanitization
2. **Safety Guardrails** intercept emergencies or diagnosis requests instantly.
3. Query is embedded -> **FAISS** retrieves top-K relevant medical documents.
4. LangChain assembles Prompt + History + Context -> Streams to **LLM**.
5. Output is validated -> Disclaimers & Citations appended -> Displayed to User.

5: Challenges & Solutions

Challenge 1: Preventing Medical Diagnoses
Solution: Implemented a multi-layered safety approach. Static regex checks block explicit requests instantly, while the LLM's system prompt handles nuanced boundary cases.

Challenge 2: PDF Parsing Quality
Solution: Standard text loaders break on medical tables and multi-column PDFs. Integrated `PyMuPDF` to implement layout-aware extraction and markdown table generation.

Challenge 3: Handling Off-Topic Queries gracefully
Solution: Initially tried keyword matching, but it was too fragile. Switched to a dynamic approach where the LLM's system prompt dictates scope, allowing meta-conversational queries (e.g., "What was my last question?") to pass naturally while blocking non-health topics.